CODE2DAY · RAMCOAD · COMBINED PLATFORM COST STUDY

Code2Day + MentorMind: Cost-Minimised Deployment

Combined monthly cost for **both products on one shared AWS estate** across **five scenarios** — one budget-capped, four capacity-defined — modelled against an **8-hour daily usage window**. Every reasonable cost lever applied.

TIER 0 · BUDGET-CAPPED AT ₹15,000/MONTH

₹14,927/mo

$175.62 · ₹73 under budget

Handles **~1,000 daily active users**, **~300 peak concurrent**, **~2,000 registered** — with 44% CPU headroom at design load. Everything is deployed and every AI feature works. **The binding constraint is the AI budget, not the servers.**

₹7.46 / student · ₹14.93 / DAU

TIER 1 · 3,000 REG · 3,000 CONCURRENT

₹76,500/mo

$900/mo · 100% concurrency · 3,000 DAU
Every student online at once — a synchronised-event shape

₹25.50 / student

TIER 2 · 10,000 REG · 3,000 CONCURRENT

₹1,46,965/mo

$1,729/mo · 30% concurrency · ~7,000 DAU
The normal campus pattern at full roll

₹14.70 / student

TIER 3 · 10,000 REG · 2,000 CONCURRENT

₹1,21,040/mo

$1,424/mo · 20% concurrency · ~7,000 DAU
Best cost per student. Same students as Tier 2, usage spread wider

₹12.10 / student

TIER 4 · 5,000 REG · 1,000 CONCURRENT

₹68,510/mo

$806/mo · 20% concurrency · ~3,500 DAU
Smallest absolute bill of the four

₹13.70 / student

The single most useful finding across all four tiers

Cost per student is set by your concurrency ratio, not your headcount. Tier 2 and Tier 3 serve the *same 10,000 students*; the only difference is whether 3,000 or 2,000 are online at peak. That one difference is worth **₹25,925 a month — 18%**, and it is a *timetabling* decision, not a technology purchase. Staggering lab and test schedules is the cheapest optimisation available anywhere in this study. §10 models the relationship.

REPORT DATE	9 September 2026 — revision 4 (adds Tier 0 budget build §11; Tiers 3–4; §9 autoscaling; §10 proportionality)
SCOPE	**Combined:** Code2Day and MentorMind on one estate. Per-product attribution in §6
REGION	ap-south-1 (Mumbai) · FX ₹85.00 / USD
USAGE WINDOW	**8 hours/day × 22 weekdays = 176 peak-hours/month** (24% of the month) — drives the largest single saving, §3
PRICE BASIS	`m7g.2xlarge` at **$0.2333/hr ap-south-1** VERIFIED. RDS, ElastiCache and ALB are baseline published rates with RI discounts applied VERIFY IN CALCULATOR. AI rates are September 2026 list prices VERIFIED
COMPANION TO	`AWS_DEPLOYMENT_OPTIONS.md`, `PROJECT_DEPLOYMENT_PROFILE.md`, `MENTORMIND_MODULE_COST_BREAKDOWN.md`

1 Summary

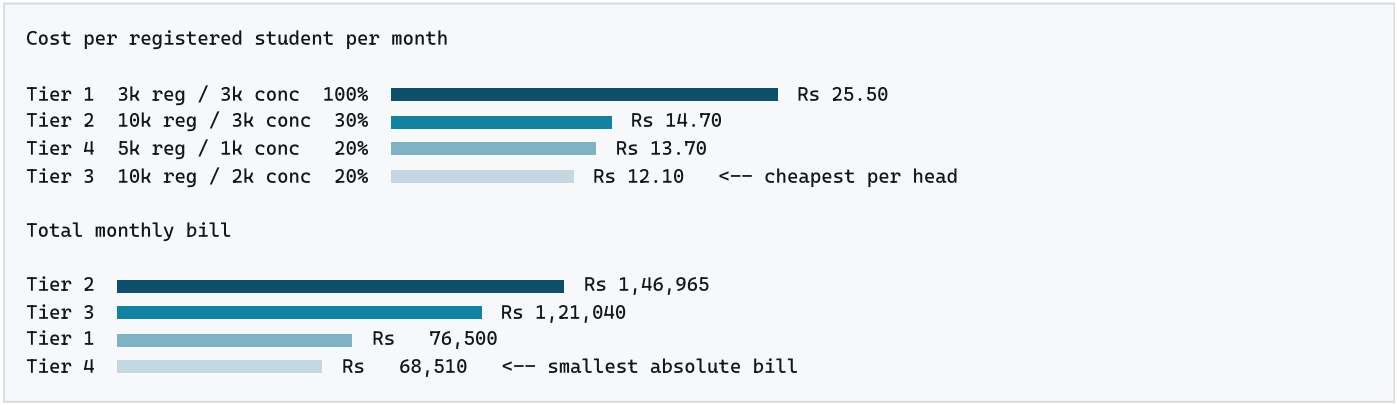
TABLE 1 — COMBINED MONTHLY COST, ALL FIVE TIERS

Cost centre	Tier 0 budget	Tier 1 3k/3k	Tier 2 10k/3k	Tier 3 10k/2k	Tier 4 5k/1k
Database tier (RDS + replicas + Proxy + storage)	$26	$260	$532	$283	$266
Application compute (EC2 + ALB)	$32	$256	$256	$206	$113
LLM inference (DeepSeek V3.2 via Mantle)	$50	$149	$347	$347	$174
Cache tier (ElastiCache for Valkey)	$0 on-instance	$70	$215	$215	$70
Speech-to-text (Groq Whisper turbo)	$18	$53	$123	$123	$62
Serverless execution (Lambda + SQS)	$12	$36	$91	$91	$44
Edge + storage (CloudFront, S3)	$8	$16	$70	$70	$17
Operations (CloudWatch, DNS, backup, IPv4)	$22	$45	$65	$59	$45
AI overflow allowance	$8	$15	$30	$30	$15
Total per month (USD)	**$176**	**$900**	**$1,729**	**$1,424**	**$806**
Total per month (INR)	**₹14,927**	**₹76,500**	**₹1,46,965**	**₹1,21,040**	**₹68,510**
Per registered student	**₹7.46**	**₹25.50**	**₹14.70**	**₹12.10**	**₹13.70**
Per daily active user	**₹14.93**	**₹25.50**	**₹21.00**	**₹17.29**	**₹19.57**

1.1 Scenario parameters at a glance

Tier	Registered	Peak concurrent	Conc. ratio	DAU	Activity ratio	Design req/s	₹/month	₹/student
Tier 0 budget	~2,000	~300	15%	~1,000	50%	90	**14,927**	₹7.46
Tier 1	3,000	3,000	100%	3,000	100%	900	76,500	₹25.50
Tier 2	10,000	3,000	30%	7,000	70%	900	1,46,965	₹14.70
Tier 3	10,000	2,000	20%	7,000	70%	600	1,21,040	**₹12.10**
Tier 4	5,000	1,000	20%	3,500	70%	300	68,510	₹13.70

Tier 0 is defined by its budget rather than its capacity, so it is not directly comparable with the rest: it deliberately gives up high availability and read scaling to fit ₹15,000. §11 details what it can serve, what was removed, and what the next ₹5,000 buys back. Tiers 1–4 are all built to the same "comfortable" standard described in §4.



Cheapest total and cheapest per student are different tiers

Tier 4 has the smallest bill (₹68,510) because it is the smallest deployment. **Tier 3** has the best unit economics (₹12.10/student) because it spreads a modest peak across the largest roll. Tier 1 is the worst on both counts per head — not because it is badly built, but because 100% concurrency means paying for a 3,000-user peak with only 3,000 students contributing.

2 Load model

> **These are stated assumptions, not measurements**
>
> `PROJECT_DEPLOYMENT_PROFILE.md:1530` lists **DAU and peak concurrency as "UNKNOWN — requires measurement."** Everything scales off Table 2. §8.1 gives the three commands that replace assumption with fact. Run them before committing budget.

TABLE 2 — LOAD PARAMETERS

Parameter	Basis	Tier 0	Tier 1	Tier 2	Tier 3	Tier 4
Registered students	stated (T0: derived from budget)	~2,000	3,000	10,000	10,000	5,000
Peak concurrent users	stated (T0: derived)	~300	3,000	3,000	2,000	1,000
Daily active users	T1 all by definition; T2–4 70% of roll; T0 capped by AI budget	~1,000	3,000	7,000	7,000	3,500
Usage window	stated: "8 hours or something"	8 h	8 h	8 h	8 h	8 h
Peak-hours/month	8 h × 22 weekdays	176	176	176	176	176
Peak request rate	0.2 req/s per concurrent user	60/s	600/s	600/s	400/s	200/s
Design target	×1.5 headroom for burst and node loss	90/s	900/s	900/s	600/s	300/s
Monthly audio	DAU × 40% × 3 min × 22 days	440 hr	1,320 hr	3,080 hr	3,080 hr	1,540 hr
Monthly LLM calls	after the §7.3 consolidation	38k	114k	267k	267k	133k
LLM tokens in / out	31.7M in, 16.2M out per 1,000 DAU	32M/16M	95M/49M	222M/113M	222M/113M	111M/57M

Tiers 2 and 3 share every population figure — same roll, same DAU, same AI volume. They differ only in peak concurrency, which is exactly what makes them the clean experiment for §10. Tier 1 is the outlier: 3,000 registered *and* 3,000 concurrent describes a **synchronised event** (a scheduled lab, a placement test, a timed contest), not a browsing pattern. It is the most demanding shape a platform this size can have.

3 The 8-hour window is the biggest lever

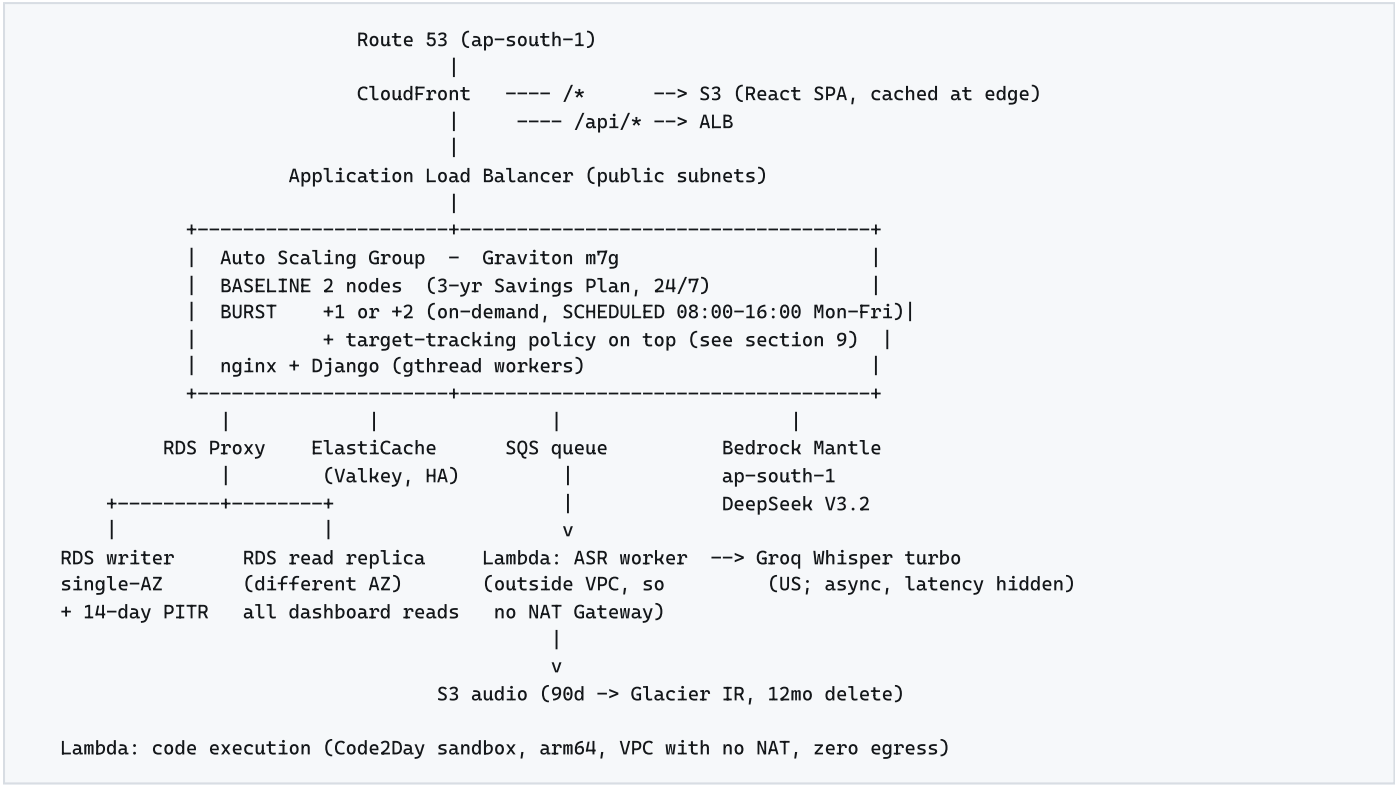
Peak capacity is needed for **176 of 730 hours a month — 24%**. Provisioning for peak around the clock wastes three-quarters of the compute spend. The chosen approach puts a **reserved baseline** underneath and buys the **burst on-demand** only during the window.

```
EC2 application fleet — m7g.2xlarge $0.2333/hr, m7g.xlarge $0.1167/hr (ap-south-1, verified)

                                          Tier 1/2      Tier 3       Tier 4
  (a) Naive: all nodes on-demand 24/7      $681          $511         $255
  (b) Savings Plan on all nodes, 24/7      $259          $194          $97
  (c) CHOSEN: SP baseline + burst for 176 h  $212        $170          $ 85

      saving of (c) over (a)               $469 (69%)    $341 (67%)   $170 (67%)
      saving of (c) over (b)               $ 47 (18%)    $ 24 (12%)   $ 12 (12%)

  Tier 1/2:  2 x m7g.2xlarge baseline (SP) + 2 burst      Tier 3: 2 baseline + 1 burst
  Tier 4:    2 x m7g.xlarge  baseline (SP) + 1 burst
```

Option (c) keeps **two nodes running at all times**, so the platform stays available and redundant outside the window, while paying on-demand rates only for capacity that exists 24% of the month. Committing a Savings Plan to the full peak fleet (option b) would waste the commitment for 554 hours a month.

> **Where the 8-hour window does *not* help**
>
> **The database and cache cannot follow it.** RDS Multi-AZ cannot be stopped, a stopped single-AZ instance restarts automatically after 7 days, and ElastiCache nodes are always-on. That is $330–$747/month of unavoidable 24/7 spend — between 36% and 52% of each tier. §4.2 covers the one genuine alternative and §9 explains why it still loses.

4 Architecture



4.1 Sizing per tier

Component	Tier 1	Tier 2	Tier 3	Tier 4
Design request rate	900/s	900/s	600/s	300/s
vCPU needed at 40 req/s per vCPU	23	23	15	8
Fleet at peak	4 × m7g.2xl 32 vCPU	4 × m7g.2xl 32 vCPU	3 × m7g.2xl 24 vCPU	3 × m7g.xl 12 vCPU
Fleet off-peak	2	2	2	2
RDS writer	r7g.large	r7g.xlarge	r7g.large	r7g.large
RDS read replicas	1	2	1	1
RDS storage (gp3)	100 GB	300 GB	300 GB	150 GB
ElastiCache nodes	2 × t4g.medium	2 × r7g.large	2 × r7g.large	2 × t4g.medium

Note how **Tier 3's database is nearly half Tier 2's** ($283 against $532) on identical data. Lower peak concurrency permits a smaller writer class and one fewer replica — the database is sized by concurrent load as much as by data volume, which is why concurrency is the dominant cost driver in §10.

4.2 Key decisions

Decision	Reasoning
Graviton throughout	~20% better price/performance, and ap-south-1 `m7g` rates undercut us-east-1. Requires dropping `sentence-transformers` — §7.3 #4
Single-AZ writer + PITR	Multi-AZ doubles the writer cost to cut failover from ~30 minutes to ~1. Defensible outside placement season; buy-back price in §5.2
Read replica in a second AZ	Serves all dashboard, leaderboard and analytics reads, and doubles as a promotion target if the writer's AZ fails
RDS Proxy	Not optional. Code2Day sets no `CONN_MAX_AGE` (`PROJECT_DEPLOYMENT_PROFILE.md:215`), so Django opens a fresh connection per request
ASR Lambda outside the VPC	Needs only S3 and the Groq API, both public. Keeping it out of the VPC means **no NAT Gateway** — saves $33–41/month
App nodes in public subnets	Public IPv4 at $3.60/node-month beats both NAT and a set of interface endpoints. Security groups admit ALB traffic only; IMDSv2 hop limit 1
Valkey, not Redis OSS	Same functionality, materially lower node and serverless rates
Provisioned RDS, not Aurora Serverless v2	See §9.3. ACU pricing carries no reserved discount, and Aurora's per-I/O charge forces I/O-Optimized at +30%

5 Detailed cost breakdown

TABLE 3 — COMPONENT-BY-COMPONENT MONTHLY COST, ALL FIVE TIERS (USD)

Component	Rate basis	T0 budget	T1 3k/3k	T2 10k/3k	T3 10k/2k	T4 5k/1k
APPLICATION COMPUTE						
EC2 baseline, 24/7	$0.2333/hr `m7g.2xl`; $0.1167 `m7g.xl`; 3-yr Savings Plan ~62% off. T0 = 1 node, T1–4 = 2 nodes	32	129	129	129	65
EC2 burst (176 h/month)	on-demand, scheduled scaling. **T0 has none** — no ALB to balance across	0	83	83	41	21
Application Load Balancer	$0.0225/hr + $0.008 per LCU-hour. **T0 has none** — CloudFront to Elastic IP direct	0	44	44	36	27
Compute subtotal		**32**	**256**	**256**	**206**	**113**
DATABASE TIER						
RDS writer, single-AZ + PITR	1-yr Reserved, partial upfront ~35% off. T0 = `db.t4g.small` burstable, 7-day PITR	20	113	227	113	113
RDS read replicas (separate AZ)	1-yr Reserved. **T0 has none** — all reads hit the writer	0	113	226	113	113
RDS Proxy	$0.015 per vCPU-hour of the writer. **T0 has none** — `CONN_MAX_AGE` instead	0	22	44	22	22
RDS storage + backups	gp3 + snapshot overage. T0 = 50 GB	6	12	35	35	18
Database subtotal		**26**	**260**	**532**	**283**	**266**
CACHE						
ElastiCache for Valkey, primary + replica	1-yr Reserved nodes. **T0 runs Redis on the app instance** — free, but not shared or durable	0	70	215	215	70
SERVERLESS EXECUTION						
Lambda — Code2Day code execution	per `AWS_DEPLOYMENT_OPTIONS.md` §5, scaled to DAU	10	31	78	78	38
Lambda — ASR worker	~10 s at 512 MB per job, $0.0000133/GB-s	2	5	12	12	6
SQS — speech job queue	first 1M requests free, then $0.40/M	0	0	1	1	0
Serverless subtotal		**12**	**36**	**91**	**91**	**44**
EDGE AND STORAGE						
CloudFront	1 TB/month perpetual free tier, then ~$0.109/GB (India)	2	3	42	42	3
S3 — SPA, resumes, media	Standard	2	5	10	10	5
S3 — audio recordings	90-day → Glacier IR, 12-month delete. **T0 deletes at 90 days**	4	8	18	18	9
Edge and storage subtotal		**8**	**16**	**70**	**70**	**17**
NETWORK AND OPERATIONS						
Public IPv4	$3.60 per address-month. **No NAT Gateway** at any tier	4	10	10	9	8
CloudWatch	7-day log retention and core alarms. **T0 = 3-day, 2 alarms**	8	20	35	30	22

Component	Rate basis	T0 budget	T1 3k/3k	T2 10k/3k	T3 10k/2k	T4 5k/1k
Route 53, ECR, AWS Backup	hosted zone, images, vault	10	15	20	20	15
Network and operations subtotal		**22**	**45**	**65**	**59**	**45**
INFRASTRUCTURE SUBTOTAL		**100**	**683**	**1,229**	**924**	**555**
AI AND INFERENCE						
DeepSeek V3.2 — Bedrock Mantle	$0.62/M in, $1.85/M out, ap-south-1 serverless	50	149	347	347	174
Groq Whisper large-v3-turbo	$0.04 per audio-hour	18	53	123	123	62
AI overflow allowance	throttle and retry months, ~7%	8	15	30	30	15
AI SUBTOTAL		**76**	**217**	**500**	**500**	**251**
TOTAL PER MONTH (USD)		**176**	**900**	**1,729**	**1,424**	**806**
TOTAL PER MONTH (INR)		**14,927**	**76,500**	**1,46,965**	**1,21,040**	**68,510**
PER REGISTERED STUDENT (INR)		**7.46**	**25.50**	**14.70**	**12.10**	**13.70**
PER DAILY ACTIVE USER (INR)		**14.93**	**25.50**	**21.00**	**17.29**	**19.57**

5.1 Savings already applied

Lever	T1	T2	T3	T4
8-hour scheduled scaling + Savings Plan on baseline only (§3)	469	469	341	170
1-year Reserved Instances on RDS and ElastiCache	156	318	201	146
Single-AZ writer instead of Multi-AZ	113	227	113	113
Whisper turbo instead of `large-v3`	94	219	219	109
Committee-of-Experts collapsed from 3 LLM calls to 1	81	189	189	95
CloudFront free tier absorbing SPA egress	62	109	109	72
Graviton across fleet, cache and database	50	85	72	40
Trimmed JSON output payloads	45	105	105	53
No NAT Gateway (ASR Lambda outside the VPC)	33	41	41	33
Audio lifecycle to Glacier IR, 12-month delete	17	40	40	20
Keyword router replacing LLM intent classification	16	37	37	19
Valkey instead of Redis OSS	12	36	36	12
Total saved against an unoptimised build	**1,148**	**1,875**	**1,503**	**882**
Unoptimised equivalent (USD)	**2,048**	**3,604**	**2,927**	**1,688**
Unoptimised equivalent (INR)	**1.74 L**	**3.06 L**	**2.49 L**	**1.43 L**

At every tier the savings exceed the bill that remains. More than half the money on this platform is saved by configuration and code decisions rather than spent with AWS. The two largest levers are free: scheduled scaling against the 8-hour window, and reserved-capacity commitments.

5.2 What was traded away, and the buy-back price

Accepted risk	T1	T2	T3	T4	Recommendation
Single-AZ writer — AZ failure means promoting the replica: tens of minutes, not one	+113	+227	+113	+113	Take Multi-AZ before placement season; single-AZ the rest of the year
No filler-word detection — Whisper strips "um"/"uh" at every size, so the counter at `communication_agent.py:76` only matches "like/so/actually/basically"	+231	+539	+539	+270	Defer unless filler accuracy becomes a defended feature
Two nodes off-peak — an unexpected evening surge queues until scaling catches up	+47	+47	+24	+12	Keep, and add the free target-tracking policy from §9.2
Public-subnet app nodes — public IPs behind security groups rather than network isolation	+33	+41	+41	+33	Keep. Strict SGs and IMDSv2 hop limit 1
7-day log retention — forensics beyond a week are gone	+15	+25	+22	+15	Keep, but export audit logs to S3 — cheap and long-lived
12-month audio retention cap	+16	+36	+36	+18	Keep. Shorter retention is better privacy as well as cheaper

6 Attribution: Code2Day versus MentorMind

The estate is shared, so most of the bill belongs to neither product alone.

TABLE 4 — COST ATTRIBUTION BY PRODUCT (USD/MONTH)

Attribution	Lines	T1	T2	T3	T4
MentorMind only	DeepSeek LLM, Groq Whisper, AI overflow, ASR Lambda, SQS, S3 audio	230 26%	531 31%	531 37%	266 33%
Code2Day only	Lambda code execution (the Judge0 replacement)	31 3%	78 5%	78 5%	38 5%
Shared platform	EC2 fleet, ALB, RDS + replicas + Proxy + storage, ElastiCache, CloudFront, S3 static, IPv4, CloudWatch, Route 53, ECR, Backup	639 71%	1,120 65%	815 57%	502 62%
Total		**900**	**1,729**	**1,424**	**806**

> **The shared majority is not MentorMind's to claim or avoid**
> Code2Day alone at these concurrencies would still need the same ALB, a comparable fleet, a database with read replicas, a cache tier and a CDN. So MentorMind's **marginal** cost is not the "MentorMind only" row — it is that row plus the share of fleet and database growth its traffic causes. MentorMind is the request-heavier of the two (dashboards, roadmaps, resume analysis, interview turns, speech submissions, against Code2Day's submit-and-poll coding loop), so a defensible planning split gives it **55% of the shared tier**:
>
> | All-in share (INR/month) | T1 | T2 | T3 | T4 |
> |---|---|---|---|---|
> | MentorMind (own + 55% shared) | 49,385 | 97,495 | 83,290 | 46,090 |
> | Code2Day (own + 45% shared) | 27,115 | 49,470 | 37,750 | 22,420 |
>
> Splitting it precisely requires the per-endpoint request mix from nginx logs (§8.1), which is unmeasured.

7 AI cost derivation

7.1 Speech-to-text

Groq `whisper-large-v3-turbo`, **$0.04 per audio-hour**, billed per second with a 10-second minimum. All recordings exceed the minimum, so the headline rate holds.

Tier	Minutes/mo	Audio-hours	Turbo	vs `large-v3`
Tier 1	79,200	1,320	$53	$147 — saves $94
Tier 2 / Tier 3	184,800	3,080	$123	$342 — saves $219
Tier 4	92,400	1,540	$62	$171 — saves $109

Two required code changes, neither costing anything. `fast_mode` defaults to `False` (`communication_agent.py:13`), nothing passes it, and `views.py:555` hardcodes `speech_best` — so every request currently runs `large-v3` at ~$0.111/hr instead of turbo at $0.04/hr. Separately, add `response_format="verbose_json"` with word-level timestamps: pacing is derived from `file_size / 4000.0` (`communication_agent.py:72`), a byte-size guess, so every words-per-minute penalty is computed off noise.

7.2 Text LLM — DeepSeek V3.2 on Bedrock Mantle

$0.62/M input, $1.85/M output. Serverless and per-token at `bedrock-mantle.ap-south-1.api.aws`, so inference stays in Mumbai. Mantle is AWS's OpenAI-compatible endpoint, so the existing OpenAI-shaped calls migrate with a base-URL and API-key change rather than a rewrite.

Tier	Calls/mo	Input	Output	Input $	Output $	Total
Tier 1	114,200	95.1 M	48.5 M	59	90	**$149**
Tier 2 / Tier 3	266,600	221.9 M	113.2 M	138	209	**$347**
Tier 4	133,300	111.0 M	56.6 M	69	105	**$174**

Volumes assume the Committee-of-Experts consolidation, a keyword router for the load balancer, and trimmed JSON responses. Un-optimised they are 1.9× higher. **Output is 60% of the LLM bill** at three times the input rate, because every agent returns JSON scores plus prose feedback — so shortening responses is worth three times as much as shortening prompts.

> **Largest single cost risk: DeepSeek reasoning mode**
>
> DeepSeek V3.2 is reasoning-capable, and reasoning tokens bill as **output at $1.85/M**. Every task here is structured scoring at `temperature=0.0` and needs no chain-of-thought. Left enabled, output tokens can rise 3–10×, adding **$180–810/month at Tier 1** and **$420–1,880 at Tiers 2 and 3** — silently, with no error and no visible product change. **Pin it off and assert it in a test.**

> **Two Mantle specifics worth knowing**
>
> **Retention.** On the Responses API, `store` defaults to `true` and AWS retains full input and output for **30 days** — here, student speech transcripts, resumes and interview answers. Data stays in ap-south-1 and is encrypted, so residency holds, but 30-day retention of identifiable student work is a consent decision. Set `store: false` per request, or set the account data-retention mode to `none`.
>
> **Quotas.** Mantle enforces *no* requests-per-minute quota — helpful for 114k–267k small calls a month — and cached input tokens do not count against the input-token quota. DeepSeek has no published TPM quota on this endpoint (only Claude Opus 4.7 does), and Provisioned Throughput is unavailable on Mantle. All four tiers need only ~12,000–28,000 input TPM sustained, which is modest, so this is a low risk at these scales.

7.3 Engineering prerequisites

The estimates assume this work is done. Items 1 and 5 are hard gates.

#	Change	Why it gates the estimate
1	**Async speech and resume pipeline** — presigned S3 upload → SQS → Lambda → poll or WebSocket	MentorMind runs `gunicorn --workers 3`; Code2Day runs 12 sync workers and already "collapses at ~20 concurrent submits" (profile §1017). Synchronous AI calls starve logins platform-wide
2	**Point the OpenAI SDK at Mantle** — new base URL, Bedrock API key, IAM `bedrock-mantle:CreateInference`	Verify `response_format` with one test call first; it should be native
3	**Collapse the Committee-of-Experts** 3 calls → 1 (`communication_agent.py:125`, `chat_practice:297`)	Worth $81–189/month and cuts speech latency 3×
4	**Drop `sentence-transformers` / `torch`**	`torch==2.10.0+cpu` ships x86_64 wheels only; every Graviton saving depends on removing it. Used in one function that already has a Jaccard fallback
5	**Read routing** — `DATABASE_ROUTERS` plus `.using()`	**Hard gate.** None exists today, so $113–226/month of read replicas do nothing until it is written
6	**Materialise campus rank** into Redis or a summary table	Three distinct-count joins in Python per dashboard load, on the writer
7	**Set `CONN_MAX_AGE`** and add RDS Proxy	A fresh connection per request exhausts the database before the instance saturates
8	**`FileBasedCache` → ElastiCache; `FileSystemStorage` → S3**	Progress tracking returns wrong data across containers; audio and resumes are lost on redeploy
9	**Fix `CORS_ALLOW_ALL_ORIGINS = True`** (`main/settings.py:154`)	Overrides the allowlist above it, with credentials enabled. Must precede public exposure
10	**Make `_fallback_transcript()` fail loudly** (`communication_agent.py:377`)	Returns a hardcoded sentence that is then scored as real speech. Throttling makes this common at scale
11	**Rate-limit `/api/communication/speaking/` per user**	An unthrottled endpoint that bills Groq per upload makes the AI bill attacker-controlled
12	**RAG → `pgvector` with an index** (`core/rag/vector_store.py:47`)	Pulls every chunk into Python per query, on the writer

Also still required from `AWS_DEPLOYMENT_OPTIONS.md` Phase 0: the unauthenticated `DROP DATABASE` endpoint, unauthenticated code execution, and the forgeable password-reset token. **Nothing here is safe to expose publicly until those are closed.**

8 Verification and sensitivity

8.1 Measure these three things first

```
-- 1. Daily active users
SELECT count(*) FROM student_profiles WHERE last_login_on = CURRENT_DATE;

-- 2. Peak concurrency and the real shape of the usage window
awk '{print $4}' code2day.access.log | uniq -c | sort -rn | head -20

-- 3. Real token counts — instrument generate_completion() for one week
```

The second decides *which tier you are*, which is a ₹78,455/month spread across this report. The third is the most valuable per hour spent, because every LLM figure rests on estimated prompt and response sizes.

8.2 Sensitivity

If this assumption is wrong	T1	T2	T3	T4
Requests per concurrent user is 0.4/s, not 0.2/s	+$310	+$420	+$330	+$180
Voice adoption is 100% of DAU, not 40%	+$300	+$700	+$700	+$350
Token counts per call are 2× the estimate	+$149	+$347	+$347	+$174
DeepSeek reasoning mode left enabled	+$180–810	+$420–1,880	+$420–1,880	+$210–940
ap-south-1 carries a 20% uplift on AWS rates	+$137	+$246	+$185	+$111
Usage window is 12 hours, not 8	+$41	+$41	+$21	+$10
Usage window is 24/7 (no scheduled scaling possible)	+$47	+$47	+$24	+$12
Read routing never implemented	Replicas become dead spend; the writer must scale vertically instead			

The usage window is cheap to get wrong. Even at full 24/7 operation the fleet adds only $12–47/month, because the cheapest 24/7 option is simply to put the whole fleet on the Savings Plan. The expensive error is under-estimating requests per concurrent user, which drives the fleet and the database together.

8.3 Unverified inputs

Item	Status and action
DAU, peak concurrency, usage-window shape	**UNKNOWN** Profile §1530. Run §8.1
`m7g.2xlarge` ap-south-1 rate	**VERIFIED** $0.2333/hr
RDS, ElastiCache, ALB ap-south-1 rates	**BASELINE + RI DISCOUNT** Confirm in the AWS Pricing Calculator; the companion document warns aggregators disagree on Mumbai
DeepSeek V3.2 price in ap-south-1	**US RATE** $0.62/$1.85 published for US East/West. Confirm in the Bedrock console
Reasoning-mode default on Mantle	**UNCONFIRMED** The largest cost swing here. Verify, then pin
`response_format` on Mantle	**INFERRED** Mantle serves the OpenAI Chat Completions API. One test call decides whether §7.3 #2 is trivial
40 requests/second per vCPU	**ESTIMATED** Load-test with k6 or Locust from inside the Region, per profile §2271
Audio bitrate ~1 MB/min	**ASSUMED** Measure real `SpeakingSession.audio_file` sizes
CloudFront egress volume	**ESTIMATED** Depends on SPA bundle size and cache hit ratio. Tiers 1 and 4 sit inside the free tier, so error there is cheap
`torch` on aarch64	**STATED AS FAILING** Confirm with a `pip install` on a Graviton instance

> **Two operational notes carried forward**
>
> **Open a Paid-plan AWS account.** Since 15 July 2025 the Free plan auto-closes after six months or when credits run out (`AWS_DEPLOYMENT_OPTIONS.md` §8 #1) — a data-loss event waiting to happen for a college. Institutional AWS Educate / Academy credits are worth applying for and should be treated as a discount on a real budget, never as the budget.
>
> **Set AWS Budgets and a Groq spend cap on day one**, with a budget action on the execution path. Neither AWS nor Groq offers a true hard ceiling.

9 Can it just autoscale? — the honest answer

Mostly yes, and much of this platform already does. But "autoscale everything" is not the cheapest answer, and for one specific reason that matters here: **the discounts that make this budget work are only available if you commit to capacity in advance, and autoscaling by definition does not commit.**

9.1 What can and cannot autoscale

Layer	Autoscales?	Mechanism	Reaction	Cost effect
CloudFront, S3	FULLY	pay-per-use, invisible	instant	Already optimal
ALB	FULLY	LCU billing, invisible	instant	Already optimal
Lambda (code exec + ASR)	FULLY	per-invocation, scale to zero	ms warm	Already optimal
SQS	FULLY	pay-per-request	instant	Already optimal
Bedrock Mantle + Groq	FULLY	per-token / per-audio-hour	instant	Already optimal — and this is 26–37% of the bill
EC2 fleet (ASG)	YES	target tracking on CPU or `RequestCountPerTarget`	**2–5 min**	Works well, but burst capacity forfeits the Savings Plan discount
RDS storage	YES	gp3 storage autoscaling	minutes	Enable it; no downside
ElastiCache nodes	PARTLY	cluster-mode auto scaling on CPU/memory	minutes	Adds cluster-mode complexity for modest gain
RDS read replicas	NO	manual add/remove only	—	Aurora has reader auto-scaling; plain RDS does not
RDS writer compute	NO	manual resize or Blue/Green	downtime	**Must be sized for peak.** This is the one layer that forces up-front capacity decisions
Aurora Serverless v2	YES	0.5–256 ACU, per-second, can pause at 0	seconds	True DB autoscaling — but see §9.3
ElastiCache Serverless	YES	GB-hour + ECPU	instant	Higher unit rate; lost to nodes at $191 vs $70 for Tier 1

9.2 The recommended answer: schedule *and* autoscale

These are not alternatives. Use both, for different jobs.

- **Scheduled scaling handles what you already know.** A college timetable is the most predictable load pattern that exists — classes start at fixed times on fixed days. Scaling up at 07:45 means capacity is *already warm* when the bell rings. Reactive autoscaling, by contrast, needs 2–5 minutes to boot, health-check and warm an instance, so it only starts adding capacity *after* a queue has formed. For Tier 1's synchronised pattern — 3,000 students logging in when a test opens, inside a minute — reactive scaling alone would guarantee a bad first two minutes for everyone.
- **Target-tracking autoscaling handles the surprises** on top of that floor: an unexpected evening surge, a viral assignment, a deploy that doubles CPU. **Add it — it is free**, it only costs money in the hours it actually fires, and it removes the risk that a wrong schedule becomes an outage.
- **Reserve the always-on baseline.** The 2-node floor runs 730 hours a month either way, so a 3-year Savings Plan on it is free money — 62% off capacity you were going to buy anyway. Autoscaling above the floor stays on-demand, which is the correct trade because you cannot predict it.

That combination is precisely what §3 costs. The only change this section recommends is **adding the target-tracking policy alongside the schedule**, which costs nothing and is listed in §5.2 as the mitigation for running two nodes off-peak.

9.3 Why fully-serverless is more expensive here

Reason	Effect
No reserved discount on serverless capacity	Savings Plans and Reserved Instances give 35–62% off. Aurora ACUs and ElastiCache Serverless have no equivalent, though AWS Database Savings Plans (December 2025) now offer up to 30% on some serverless usage — a partial exception worth checking
Higher unit rates	Aurora at $0.12/ACU-hour running at peak exceeds an RI-discounted provisioned instance. ElastiCache Serverless costed $191/month against $70 for nodes at Tier 1
Aurora charges per I/O	$0.20 per million read/write I/Os. At these request rates that could add $500–2,000/month, so I/O-Optimized becomes mandatory — at +30% on ACU cost
Scale-up lag against synchronised load	2–5 minutes for EC2. A bell-schedule spike arrives in under one

Costed side by side for the database, the chosen configuration wins at every tier:

Database option	T1	T2	T3	T4
RDS provisioned + 1-yr RI CHOSEN	$260	$532	$283	$266
Aurora Serverless v2, I/O-Optimized	$551	$938	$680	$430
Aurora Serverless v2, Standard storage	Cheaper on ACU but exposed to $500–2,000/month of I/O charges — not viable			

Revisit Aurora only if measured off-peak load turns out to be genuinely near-idle and the peak ACU requirement proves lower than the 8–16 ACU assumed here. If the platform ever moves to genuinely unpredictable, spiky, multi-tenant traffic — several colleges on different timetables — the calculus flips and Aurora Serverless v2 becomes the right answer.

> **The short version**
>
> The layers that benefit most from autoscaling — AI inference, code execution, speech processing, CDN — **already autoscale perfectly and cost nothing when idle.** That is 26–37% of the bill with zero waste. The layer that cannot autoscale cheaply is the **database**, and that is also the largest line item at every tier. So the answer to "can we just autoscale everything?" is: *you already do, everywhere it pays.* What remains is a database that must be sized for peak, and a fleet where a reserved floor plus a scheduled burst beats pure reactive scaling on both cost and latency.

10 The proportionality model — what actually drives the bill

With four scenarios costed, the underlying relationship becomes visible. Two independent ratios determine cost per student, and only one of them is about how many students you have.

10.1 The two drivers

Driver	Definition	Sets the cost of	Behaviour
Concurrency ratio	peak concurrent ÷ registered	EC2 fleet, ALB, database class, replica count, operations	**Step-shaped.** Crosses instance-class boundaries, so cost jumps rather than glides
Activity ratio	daily active users ÷ registered	LLM tokens, audio hours, storage, Lambda, database size	**Linear.** Every extra active student costs the same as the last

TABLE 5 — CAPACITY-DRIVEN VERSUS POPULATION-DRIVEN COST, PER STUDENT

Tier	Conc. ratio	Activity ratio	Capacity-driven ₹/student	Population-driven ₹/student	Total ₹/student
Tier 1 3k/3k	100%	100%	₹8.53	₹16.97	₹25.50
Tier 2 10k/3k	30%	70%	₹2.73	₹11.97	₹14.70
Tier 3 10k/2k	20%	70%	₹2.25	₹9.85	**₹12.10**
Tier 4 5k/1k	20%	70%	₹2.69	₹11.02	₹13.70

Capacity-driven cost per student collapses from **₹8.53 to ₹2.25** — a 74% fall — as the concurrency ratio drops from 100% to 20%. Population-driven cost falls far less, because it is genuinely proportional to use. **The concurrency ratio is the lever; the activity ratio is mostly a fact about your students.**

10.2 The three things this tells you

> **1. Stagger the timetable. It is the cheapest optimisation in this report.**
> Tiers 2 and 3 serve identical students with identical AI volume. The only difference is 3,000 versus 2,000 peak concurrent — and that is worth **₹25,925/month, or ₹3.11 lakh a year.** It buys a smaller database writer, one fewer read replica, one fewer application node and a smaller load balancer. Spreading lab sessions and assessment windows across more of the day is a scheduling decision that costs nothing and outperforms every technical lever except the 8-hour window itself.

> **2. More students on the same peak is always cheaper per head**
> Tier 3 has twice Tier 4's roll but only 1.8× the bill, so it is 12% cheaper per student. Tier 2 has 3.3× Tier 1's roll for 1.9× the bill — 42% cheaper per student. If the roll is going to grow, **the marginal student is far cheaper than the average student**, so build the peak once and fill it.

> **3. There is a floor, and it is around ₹10–12 per student**
> Population-driven cost never falls below about **₹9.85/student** in these scenarios, because AI tokens, audio hours and storage are genuinely proportional to use. No architecture removes that. Getting below it means changing what the product does per student — the daily-exercise cap in the ₹7,000 analysis of `MENTORMIND_MODULE_COST_BREAKDOWN.md` §9.5 is exactly that trade.

10.3 Estimating a scenario not costed here

The relationship is not a clean formula — instance classes are step functions, and several components (cache size, database class) straddle both drivers. But the four points bracket the useful range, and this procedure gets within roughly 15%:

```
1. Find the costed tier whose PEAK CONCURRENCY is closest to yours.
   That fixes the capacity half — fleet, ALB, DB class, replicas, ops.
   -> 1,000 conc: use Tier 4's $158    2,000 conc: Tier 3's $265
      3,000 conc: Tier 1/2's $301–321

2. Scale the population half by YOUR DAU against that tier's DAU.
   Population-driven = DB + cache + serverless + edge/storage + AI
   -> Tier 1 $599 @ 3,000 DAU   Tier 2/3 $1,408/$1,159 @ 7,000 DAU
      Tier 4 $648 @ 3,500 DAU
   Roughly $0.17 per DAU per month, plus a database-class step.

3. Add the two. Then add 15% for the step you are probably straddling.

4. Sanity-check against Rs 12–26 per registered student per month.
   Anything outside that band means an assumption is wrong.
```

Worked example — 20,000 registered, 3,000 concurrent, 14,000 DAU: capacity half ≈ $321 (Tier 2's peak); population half ≈ $1,408 × (14,000 ÷ 7,000) = $2,816; total ≈ $3,137, plus 15% ≈ **$3,600/month ≈ ₹3.06 lakh**, or **₹15.30 per student**. That sits inside the expected band, and the concurrency ratio of 15% suggests it would come in nearer ₹13 with the database sized to the real read pattern.

11 Tier 0 — the ₹15,000 budget build

This tier inverts the question: instead of "what does capacity X cost", it asks **"what can ₹15,000 a month actually serve, with everything deployed?"** The answer is a genuinely complete platform — every feature works, every AI agent runs — for a real but clearly bounded population.

What ₹15,000/month buys

Capacity	Value	Basis
Daily active users	**~1,000**	The real ceiling. Set by the AI budget, not the servers
Peak concurrent users	**~300**	Comfortable, with 44% CPU headroom at the 90 req/s design target
Registered students	**~2,000**	At a 50% activity ratio. Could be 3,000 at 33% activity, or 1,430 at 70%
Actual monthly cost	₹14,927	$175.62 — ₹73 under budget
Cost per student	₹7.46	Lowest per-student figure in this report — because activity is low, not because it is efficient
Cost per daily active user	₹14.93	The honest unit metric for this tier

11.1 The binding constraint is AI spend, not compute

This is the most important thing to understand about Tier 0. The single `m7g.xlarge` (4 vCPU) has capacity for **160 req/s**, which is roughly **800 concurrent users** in raw terms. It is not the limit. The limit is that every active student costs about **$0.079/month in AI and Lambda**, and after fixed infrastructure only $88 remains for that.

```
Budget allocation — split by FIXED vs VARIABLE
(a different cut from Table 3, which groups Lambda under infrastructure)

  Fixed:    compute, database, cache, edge, operations        $ 88   Rs  7,480
  Variable: AI + Lambda, at $0.079 per DAU per month          $ 88   Rs  7,447
                                                             ----   ---------
  Total                                                      $176   Rs 14,927

  $88 / $0.079 per DAU  =  ~1,100 DAU ceiling
                           (1,000 used, leaving margin for the overflow allowance)

Compute capacity at 1 x m7g.xlarge (4 vCPU, 40 req/s per vCPU):

    200 concurrent -> design  60 req/s of 160    62% headroom
    300 concurrent -> design  90 req/s of 160    44% headroom   <-- chosen
    400 concurrent -> design 120 req/s of 160    25% headroom
    500 concurrent -> design 150 req/s of 160     6% headroom   <-- too tight
```

Practical implication: if you enforce a per-student daily AI quota — say two graded exercises instead of three — the variable cost drops to about $0.055/DAU and the same budget serves **~1,600 DAU**. Tier 0's headline number is a policy choice as much as a hardware one.

11.2 What is cut, and what it costs

Tiers 1–4 are built to the "comfortable" standard in §4. These are the specific things Tier 0 gives up to reach ₹15,000, each with the price of buying it back.

#	Cut	What it actually means	Buy back
1	**No load balancer**	**Single point of failure.** CloudFront points at one Elastic IP. If the instance dies, the site is down until it recovers — a few minutes with EC2 auto-recovery, longer if the failure is not hardware. No health checks, no graceful draining on deploy	+$44 ₹3,740
2	**No horizontal scaling**	You cannot add a second application node without also adding the ALB. Growth past ~400 concurrent means re-architecting, not just resizing. There is also no scheduled burst — so the §3 saving does not apply here	+$65 with node
3	**No read replica**	Every dashboard, leaderboard and analytics query competes with writes on one burstable instance. The read-routing work in §7.3 #5 has nowhere to route to, so it can be deferred — but it is then the first thing needed on growth	+$113 ₹9,605
4	**No RDS Proxy**	`CONN_MAX_AGE` *must* be set instead (§7.3 #7) — a free code change, but now mandatory rather than advisable. Without it the database runs out of connections well before the CPU is busy	+$22 ₹1,870
5	**Redis on the app instance**	Competes with Django for RAM and CPU. Lost entirely when the instance is replaced, so sessions drop and every student is logged out on deploy. Cannot be shared if a second node is ever added	+$70 ₹5,950
6	**Burstable database** (`db.t4g.small`)	Runs on CPU credits. Sustained load past the baseline throttles hard, and 2 GB of RAM means a small PostgreSQL cache and more disk I/O. Adequate for 300 concurrent; unpleasant above it	+$93 to r7g.large
7	**Single-AZ, no Multi-AZ**	An availability-zone failure means restoring from snapshot — hours, not minutes. With no read replica there is not even a promotion candidate	+$20 ₹1,700
8	**7-day PITR** instead of 14	Recovery window halved. Adequate, but a problem discovered after a fortnight is unrecoverable	+$4
9	**3-day CloudWatch retention, 2 alarms**	Effectively no forensics. An incident investigated on Monday about Friday's outage has no logs. Export critical audit logs to S3 — cheap and long-lived	+$12
10	**90-day audio deletion** (no Glacier tier)	No session replay beyond a term. Arguably better for privacy, but students lose their own history	+$4
11	**No synchronised-event capacity**	**Tier 1's pattern is impossible here.** A "everyone opens the test at 10:00" moment at 1,000 students would queue badly. Assessments must be staggered, and that is a hard operational constraint, not a preference	—
12	**Per-student AI quota required**	With only $88/month of AI headroom, an unthrottled usage spike blows the budget in days. §7.3 #11's rate limit stops being a security nicety and becomes a financial control	—

11.3 What is *not* cut

Worth stating plainly, because the list above is long. Tier 0 keeps everything that makes the platform work correctly:

- **44% CPU headroom at design load** — this is a comfortable single node, not a saturated one. Latency targets in the earlier tiers' terms are met at 300 concurrent.
- **Managed PostgreSQL with point-in-time recovery.** Self-hosting Postgres on the app instance would have saved $26 more; losing PITR on student placement records is not a reasonable trade at any budget, so it was not taken.
- **The asynchronous speech pipeline.** S3 → SQS → Lambda stays, because it costs almost nothing ($12) and is the difference between working and collapsing.
- **Every AI feature, at full quality.** Same DeepSeek V3.2 on Mantle, same Whisper turbo, same models as every other tier. Nothing is downgraded to a cheaper model.
- **CloudFront edge caching, Graviton compute, and all twelve §5.1 cost levers.**

11.4 The upgrade path

The next money is worth spending in a specific order.

Step	Add	Cost/mo	Running total	Buys
Tier 0	as costed above	₹14,927	₹14,927	~1,000 DAU, ~300 concurrent, single point of failure
+1	**ALB + a second node during the window**	+₹5,485	**₹20,412**	**Real high availability.** The single most valuable next rupee — removes the SPOF and enables the §3 scheduled-burst saving
+2	ElastiCache (2 nodes)	+₹5,950	₹26,362	Sessions survive deploys; cache shared across both nodes. Required by step 1 to be useful
+3	Read replica + RDS Proxy	+₹11,475	₹37,837	Dashboards stop competing with writes; read routing pays off; a promotion target exists
+4	Non-burstable database (`r7g.large`)	+₹7,905	₹45,742	No more CPU-credit throttling. At this point you are at Tier 4's shape (₹68,510) minus its larger AI volume

> **If only ₹5,000 more is available, spend it on step 1**
> Steps 2–4 improve performance. **Step 1 is the only one that changes whether the platform survives a single instance failure.** At ₹20,412/month you have a genuinely production-shaped deployment; at ₹14,927 you have a working one that will eventually have a bad day. For a platform holding placement records during placement season, that distinction is worth more than any latency improvement below it.

12 Sources

AWS documentation. Responses API — bedrock-mantle endpoint (ap-south-1 availability, authentication, `store` retention) · Quotas for the bedrock-mantle endpoint (input/output TPM, no RPM enforcement, no Provisioned Throughput) · DeepSeek V3.2 model card · Bedrock pricing · Lambda pricing · RDS for PostgreSQL pricing · EC2 on-demand pricing · ElastiCache Serverless · Aurora Serverless v2 scaling to zero.

Rate references. m7g.2xlarge ap-south-1 — $0.2333/hr · DeepSeek V3.2 on Bedrock — $0.62/$1.85 per 1M · Groq pricing 2026 — Whisper turbo $0.04/audio-hour · Aurora Serverless v2 — $0.12/ACU-hr and when it saves money · ElastiCache pricing — Valkey vs Redis OSS, serverless vs node · RDS db.r7g rates · ElastiCache node rates · Deepgram Nova-3 — $0.0043/min.

Repository documents. `AWS_DEPLOYMENT_OPTIONS.md` §§1, 5, 6, 7, 8, 9 · `PROJECT_DEPLOYMENT_PROFILE.md` §§215, 286, 362, 1017, 1530, 2271 · `MENTORMIND_MODULE_COST_BREAKDOWN.md` §§2, 4, 5, 7, 9, 10.

Code2Day + MentorMind: Cost-Minimised Deployment — combined platform cost study across five scenarios — one budget-capped, four capacity-defined — 8-hour daily usage window. Revision 4, prepared 9 September 2026 for ap-south-1 at ₹85/USD. **Re-verify all AWS figures in the AWS Pricing Calculator before commitment.** Companion to `AWS_DEPLOYMENT_OPTIONS.md` , `PROJECT_DEPLOYMENT_PROFILE.md` and `MENTORMIND_MODULE_COST_BREAKDOWN.md` .